

ITEC610 – INTRODUCTION TO DATA SCIENCE WITH PYTHON

Assessment 2 - Data preparation tasks

Semester 2, 2024

Submitted to:

Vu Nguyen

Submitted by:

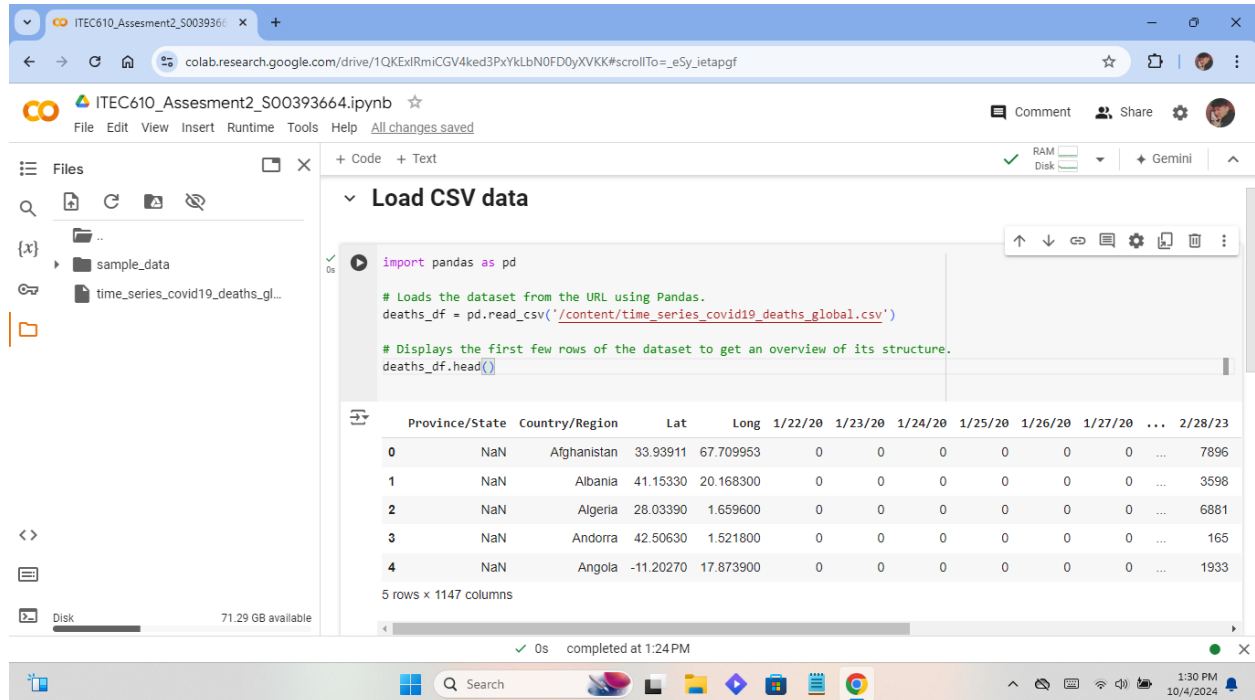
Amir Tamang

ID: S00393664

Email: S00393664@myacu.edu.au

Date of Submission: 04/10/2024

1. Load CSV data:



The screenshot shows a Google Colab notebook titled "ITEC610_Assessment2_S00393664.ipynb". The code cell "Load CSV data" contains the following Python code:

```
import pandas as pd

# Loads the dataset from the URL using Pandas.
deaths_df = pd.read_csv('/content/time_series_covid19_deaths_global.csv')

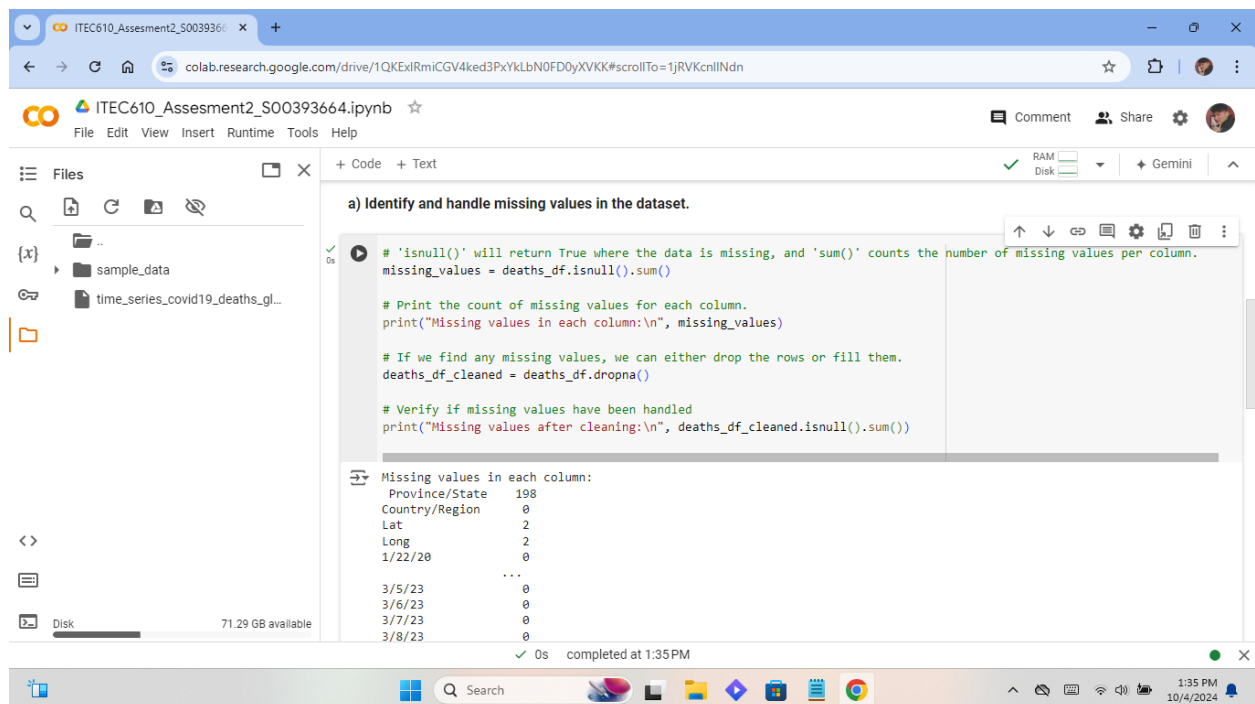
# Displays the first few rows of the dataset to get an overview of its structure.
deaths_df.head()
```

The output of the code is a preview of the first 5 rows of the dataset:

	Province/State	Country/Region	Lat	Long	1/22/20	1/23/20	1/24/20	1/25/20	1/26/20	1/27/20	...	2/28/23
0	NaN	Afghanistan	33.93911	67.709953	0	0	0	0	0	0	...	7896
1	NaN	Albania	41.15330	20.168300	0	0	0	0	0	0	...	3598
2	NaN	Algeria	28.03390	1.659600	0	0	0	0	0	0	...	6881
3	NaN	Andorra	42.50630	1.521800	0	0	0	0	0	0	...	165
4	NaN	Angola	-11.20270	17.873900	0	0	0	0	0	0	...	1933

The output indicates 5 rows and 1147 columns. The status bar shows "completed at 1:24 PM".

- Identify and handle missing values in the dataset.



The screenshot shows the same Google Colab notebook, now with a new code cell titled "a) Identify and handle missing values in the dataset." The code is as follows:

```
# 'isnull()' will return True where the data is missing, and 'sum()' counts the number of missing values per column.
missing_values = deaths_df.isnull().sum()

# Print the count of missing values for each column.
print("Missing values in each column:\n", missing_values)

# If we find any missing values, we can either drop the rows or fill them.
deaths_df_cleaned = deaths_df.dropna()

# Verify if missing values have been handled
print("Missing values after cleaning:\n", deaths_df_cleaned.isnull().sum())
```

The output shows the count of missing values for each column:

```
Missing values in each column:
Province/State    198
Country/Region     0
Lat                2
Long               2
1/22/20           0
...
3/5/23            0
3/6/23            0
3/7/23            0
3/8/23            0
```

The status bar shows "completed at 1:35 PM".

- Remove duplicate entries if any.

The screenshot shows a Google Colab notebook titled "ITEC610_Assessment2_S00393664.ipynb". The code in the cell is as follows:

```
# 'duplicated()' will return True for duplicate rows, and 'sum()' counts them.
duplicate_rows = deaths_df_cleaned.duplicated().sum()

# Prints the number of duplicate rows.
print("Number of duplicate rows:", duplicate_rows)

# If there are duplicate rows, remove them using 'drop_duplicates()'.
deaths_df_cleaned = deaths_df_cleaned.drop_duplicates()

# Verifies if duplicates have been removed by checking for duplicates again.
print("Duplicate rows after cleaning:", deaths_df_cleaned.duplicated().sum())
```

The output of the code is:

```
Number of duplicate rows: 0
Duplicate rows after cleaning: 0
```

The notebook interface shows a file explorer on the left with a folder named "sample_data" containing a file "time_series_covid19_deaths_gl...". The bottom status bar indicates "0s completed at 1:36 PM".

- Convert the date column to a consistent date format (e.g., YYYYMM-DD)

The screenshot shows a Google Colab notebook titled "ITEC610_Assessment2_S00393664.ipynb". The code in the cell is as follows:

```
# First, I gathered the columns that contain the date information.
date_columns = deaths_df_cleaned.columns[4:] # Dates start from the 5th column onwards.

# We'll 'melt' the dataframe to convert date columns into rows, so we have a proper date column.
deaths_df_melted = deaths_df_cleaned.melt(id_vars=['Province/State', 'Country/Region', 'Lat', 'Long'],
var_name='Date', value_name='Deaths')

# Convert the 'Date' column to datetime format to standardize it.
deaths_df_melted['Date'] = pd.to_datetime(deaths_df_melted['Date'], format='%m/%d/%y')

# Display the cleaned dataset to verify the changes.
deaths_df_melted.head()
```

The output of the code is a DataFrame with the following columns: Province/State, Country/Region, Lat, Long, Date, Deaths. The first few rows are:

	Province/State	Country/Region	Lat	Long	Date	Deaths
0	Australian Capital Territory	Australia	-35.4735	149.0124	2020-01-22	0
1	New South Wales	Australia	-33.8688	151.2093	2020-01-22	0
2	Northern Territory	Australia	-12.4634	130.8456	2020-01-22	0
3	Queensland	Australia	-27.4698	153.0251	2020-01-22	0
4	South Australia	Australia	-34.0285	138.0007	2020-01-22	0

The notebook interface shows a file explorer on the left with a folder named "sample_data" containing a file "time_series_covid19_deaths_gl...". The bottom status bar indicates "0s completed at 1:38 PM".

- Save the cleaned dataset to a new CSV file.

The screenshot shows a Google Colab notebook titled "ITEC610_Assesment2_S00393664.ipynb". The left sidebar displays the file explorer with a folder named "sample_data" containing two files: "cleaned_covid19_deaths.csv" and "time_series_covid19_deaths_gl...". The main code cell, titled "d) Save the cleaned dataset to a new CSV file.", contains the following Python code:

```
# This will save the dataframe to a file named 'cleaned_covid19_deaths.csv'.
deaths_df_melted.to_csv('cleaned_covid19_deaths.csv', index=False)

# Print confirmation that the file has been saved.
print("Cleaned dataset has been saved as 'cleaned_covid19_deaths.csv'.")
```

The output of the code cell shows a confirmation message: "Cleaned dataset has been saved as 'cleaned_covid19_deaths.csv'." The bottom status bar indicates the notebook is completed at 1:39 PM.

2. Display the first 5 rows of the loaded data and do a summary of the data:

The screenshot shows the same Google Colab notebook, now with a new code cell titled "2. Display first 5 rows of the loaded data and do a short summary about the data". The code in this cell is:

```
print("First 5 rows of the cleaned dataset:")
print(deaths_df_melted.head())

# This dataset represents the number of deaths due to COVID-19, as recorded worldwide.
# The significant columns for geographic data are "Province/State," "Country/Region," "Lat," and "Long.".
# The "Deaths" column also has the accumulated total of deaths and the "Date" column reflects the reporting day.
# Dates are in standard format, with no missing or duplicate values.
```

The output of the code cell displays the first 5 rows of the cleaned dataset in a table format:

	Province/State	Country/Region	Lat	Long	Date
0	Australian Capital Territory	Australia	-35.4735	149.0124	2020-01-22
1	New South Wales	Australia	-33.8688	151.2093	2020-01-22
2	Northern Territory	Australia	-12.4634	130.8456	2020-01-22
3	Queensland	Australia	-27.4698	153.0251	2020-01-22
4	South Australia	Australia	-34.9285	138.6007	2020-01-22

Below the table, the output shows the "Deaths" column for the first 5 rows:

```
Deaths
0      0
1      0
2      0
3      0
```

The bottom status bar indicates the notebook is completed at 1:46 PM.

3. Calculate the mean and median of the daily cases.

The screenshot shows a Google Colab notebook titled "ITEC610_Assesment2_S00393664.ipynb". The left sidebar displays a file explorer with a folder named "sample_data" containing two files: "cleaned_covid19_deaths.csv" and "time_series_covid19_deaths_gl...". The main code cell, titled "3. Calculate the mean and median of the daily cases.", contains the following Python code:

```
# Since the 'Deaths' column contains cumulative numbers, we need to calculate the daily death cases first.
# To do this, we'll group the data by 'Country/Region' and 'Date' to get the daily deaths for each country.

# Sort by date to make sure we calculate daily changes correctly.
deaths_df_melted = deaths_df_melted.sort_values(by=['Country/Region', 'Date'])

# Group by 'Country/Region' and 'Date', then calculate the daily increase in deaths.
deaths_df_melted['Daily_Deaths'] = deaths_df_melted.groupby(['Country/Region'])['Deaths'].diff().fillna(0)

# Calculate the mean and median of the daily death cases across all countries.
mean_daily_deaths = deaths_df_melted['Daily_Deaths'].mean()
median_daily_deaths = deaths_df_melted['Daily_Deaths'].median()

# Display the mean and median values.
print(f"Mean daily death cases: {mean_daily_deaths}")
print(f"Median daily death cases: {median_daily_deaths}")
```

The output of the code is displayed below the code cell:

```
Mean daily death cases: 0.011226124824284605
Median daily death cases: 0.0
```

The bottom status bar indicates the notebook was completed at 1:56 PM on 10/4/2024.

4. Get daily death cases worldwide.

The screenshot shows a Google Colab notebook titled "ITEC610_Assesment2_S00393664.ipynb". The left sidebar displays a file explorer with a folder named "sample_data" containing two files: "cleaned_covid19_deaths.csv" and "time_series_covid19_deaths_gl...". The main code cell, titled "4. Get Daily Death Cases Worldwide", contains the following Python code:

```
# Sum the 'Daily_Deaths' across all countries for each date.
worldwide_daily_deaths = deaths_df_melted.groupby('Date')['Daily_Deaths'].sum().reset_index()

# Display the first few rows of the worldwide daily deaths data.
print("Worldwide daily death cases (first 5 days):")
print(worldwide_daily_deaths.head())
```

The output of the code is displayed below the code cell:

```
Worldwide daily death cases (first 5 days):
   Date  Daily_Deaths
0 2020-01-22         0.0
1 2020-01-23         0.0
2 2020-01-24         0.0
3 2020-01-25         0.0
4 2020-01-26         0.0
```

The bottom status bar indicates the notebook was completed at 1:57 PM on 10/4/2024.

5. Get a daily increase in death cases.

The screenshot shows a Google Colab notebook titled "ITEC610_Assesment2_S00393664.ipynb". The left sidebar displays a file explorer with a folder named "sample_data" containing two files: "cleaned_covid19_deaths.csv" and "time_series_covid19_deaths_gl...". The main code cell, titled "5. Get Daily Increment in Deaths", contains the following Python code:

```
def calculate_daily_increase(df):  
    # 'diff()' calculates the difference between successive rows in the 'Daily_Deaths' column.  
    df['Daily_Increase'] = df['Daily_Deaths'].diff().fillna(0)  
    return df  
  
# Apply the function to the worldwide daily deaths data.  
worldwide_daily_deaths = calculate_daily_increase(worldwide_daily_deaths)  
  
# Display the first few rows to verify the calculations.  
print("Worldwide daily death increment (first 5 days):")  
print(worldwide_daily_deaths.head())
```

The output of the code is a table showing the first five rows of the "worldwide_daily_deaths" dataset:

	Date	Daily_Deaths	Daily_Increase
0	2020-01-22	0.0	0.0
1	2020-01-23	0.0	0.0
2	2020-01-24	0.0	0.0
3	2020-01-25	0.0	0.0
4	2020-01-26	0.0	0.0

The notebook interface shows the code was executed successfully at 1:59 PM on 10/4/2024.

6. Visualize the data obtained in task 5 with the library matplotlib.

The screenshot shows a Google Colab notebook titled "ITEC610_Assesment2_S00393664.ipynb". The left sidebar displays the same file explorer as in the previous screenshot. The main code cell, titled "6. Visualize the Data with Matplotlib", contains the following Python code:

```
import matplotlib.pyplot as plt  
# Plot the worldwide daily death cases.  
plt.figure(figsize=(10, 6))  
  
# Line plot for daily deaths.  
plt.plot(worldwide_daily_deaths['Date'], worldwide_daily_deaths['Daily_Deaths'], label='Daily Deaths', color='blue')  
  
# Line plot for daily death increment.  
plt.plot(worldwide_daily_deaths['Date'], worldwide_daily_deaths['Daily_Increase'], label='Daily Increment in Deaths', color='red')  
  
# Add titles and labels to the plot.  
plt.title('Worldwide Daily COVID-19 Deaths and Increments', fontsize=16)  
plt.xlabel('Date', fontsize=12)  
plt.ylabel('Number of Deaths', fontsize=12)  
plt.legend()  
  
# Display the plot.  
plt.grid(True)  
plt.show()
```

The notebook interface shows the code was executed successfully at 2:00 PM on 10/4/2024.

